EAPS507_Pandas_Data_Manipulation_InClass_Tutorial

October 14, 2026

1 EAPS 507 — Pandas for Earth Science Data Manipulation

1.1 A 2-hour guided in-class tutorial using Indiana geographic features

Dataset: `IN_Features_20140204.txt`

1.1.1 Learning objectives

By the end of this activity, you should be able to:

1. load a non-comma-delimited text file with `pandas`;
2. inspect a `DataFrame` and diagnose its structure;
3. select columns and filter observations with Boolean conditions;
4. identify and handle missing values deliberately;
5. summarize categorical and numerical data;
6. use `groupby()` and `agg()` to answer scientific questions;
7. sort data and create derived variables;
8. convert text dates into datetime objects; and
9. combine several Pandas operations to answer a question you formulate yourself.

1.1.2 How this notebook works

This is **not** a copy-and-paste exercise. Each section follows the same cycle:

Explain → instructor example → predict/test → your turn → interpret

Some cells are complete. Others contain blanks (`...`) for you to complete. Before running a **Predict first** cell, write down what you expect to happen.

Suggested pace: Sections 1–3 (45 min), Sections 4–6 (45 min), Sections 7–8 + final challenge (30 min).

1.2 0. Setup and the dataset — about 5 minutes

The file contains named geographic features and attributes such as feature class, state, county, latitude/longitude, elevation, map name, and dates. It is a **pipe-delimited** text file, so fields are separated by `|` rather than commas.

The goal today is not to memorize Pandas syntax. The goal is to learn how to move from a **data question** to an appropriate sequence of Pandas operations.

[1]:
```
import pandas as pd

# This option makes wide tables easier to inspect in Jupyter.
pd.set_option("display.max_columns", 25)
```

[2]:
```
# read_csv() can read many delimited text files, not only .csv files.
# sep="|" tells Pandas that the vertical bar separates the columns.

df = pd.read_csv("IN_Features_20140204.txt", sep="|")

df.head()
```

[2]:

	FEATURE_ID	FEATURE_NAME	FEATURE_CLASS	STATE_ALPHA	STATE_NUMERIC	COUNTY_NAME	COUNTY_NUMERIC	PRIMARY_LAT_DMS	PRIM_LONG_DMS	PRIM_LAT_DEC	PRIM_LONG_DEC	SOURCE_LAT_DMS	SOURCE_LONG_DMS	SOURCE_LAT_DEC	SOURCE_LONG_DEC	ELEV_IN_M	ELEV_IN_FT	MAP_NAME	DATE_CREATED	DATE_EDITED
0	403294	Brouilletts Creek	Stream	IN	18	Whitley	183	393617N	0872305W	39.604762	-87.384782	393919N	0874800W	39.655230	-87.799900	144.0	472.0	New Goshen	01/15/1980	11/19/2010
1	404056	Beaver Creek	Stream	IL	17	Iroquois	75	405811N	0874850W	40.969755	-87.813927	410159N	0872342W	41.033090	-87.395030	186.0	610.0	L'Erable	01/15/1980	NaN
2	406202	Clear Creek	Stream	IL	17	Edgar	45	392645N	0873001W	39.445869	-87.500300	393514N	0874118W	39.587257	-87.688360	146.0	479.0	Dennison	01/15/1980	NaN
3	406288	Coal Creek	Stream	IL	17	Edgar	45	393829N	0873006W	39.641425	-87.501687	393855N	0873750W	39.648646	-87.630580	151.0	495.0	Saint Bernice	01/15/1980	NaN
4	406345	Cole Creek	Stream	IL	17	Iroquois	75	403756N	0873331W	40.632259	-87.558636	403756N	0873331W	40.632259	-87.558636	207.0	679.0	Darrow	01/15/1980	NaN

1.2.1 Student-led test 0

Without running another command yet, inspect the first five rows above.

1. Name **two categorical** variables.
2. Name **two numerical** variables.
3. Identify one variable that appears to contain missing values.
4. Why would `pd.read_csv(..., sep="|")` work even though the filename ends in `.txt`?

Discuss your answers with a neighbor for 2 minutes.

1. Feature_Class, State_Alpha
2. Elev_in_M, Prim_Lat_Dec
3. Date_Edited
4. I believe it is because the pd.read_csv command simply doesnt read the file extension.

1.3 1. Inspecting a DataFrame — about 15 minutes

Before manipulating a dataset, learn its structure.

Useful attributes/methods:

- `df.head()` — first rows
- `df.tail()` — last rows
- `df.shape` — `(number of rows, number of columns)`
- `df.columns` — column labels
- `df.dtypes` — inferred data type of each column
- `df.info()` — compact summary including non-null counts and data types

Notice that `shape`, `columns`, and `dtypes` are **attributes**, while `head()` and `info()` are **methods**.

[3]:
```
print("Shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
```

```
Shape: (29536, 20)

Column names:
['FEATURE_ID', 'FEATURE_NAME', 'FEATURE_CLASS', 'STATE_ALPHA', 'STATE_NUMERIC',
'COUNTY_NAME', 'COUNTY_NUMERIC', 'PRIMARY_LAT_DMS', 'PRIM_LONG_DMS',
'PRIM_LAT_DEC', 'PRIM_LONG_DEC', 'SOURCE_LAT_DMS', 'SOURCE_LONG_DMS',
'SOURCE_LAT_DEC', 'SOURCE_LONG_DEC', 'ELEV_IN_M', 'ELEV_IN_FT', 'MAP_NAME',
'DATE_CREATED', 'DATE_EDITED']
```

1.3.1 Predict first

What do you expect Pandas to report for `ELEV_IN_M`, `FEATURE_NAME`, and `DATE_CREATED`?

Write your prediction before running the next cell.

I would expect that elev in m is going to be float64 as it contains numbers with decimals then both feature name and date created will be objects, feature name because it is text and date created because the date is stored and read as text unless told otherwise.

[4]:
```
df[["ELEV_IN_M", "FEATURE_NAME", "DATE_CREATED"]].dtypes
```

[4]:
```
ELEV_IN_M       float64
FEATURE_NAME     object
DATE_CREATED     object
dtype: object
```

1.3.2 Why is DATE_CREATED not automatically a date?

Pandas reads many text fields as `object` unless we explicitly tell it how to interpret them. Later, we will convert the date columns.

1.3.3 Student-led test 1

Complete the cells below. Do not ask the instructor for the answer until you have tried the documentation/help system or discussed it with a neighbor.

[5]:
```
# TEST 1A
# Display the LAST 7 records in the dataset.
# Hint: a method introduced above does this.
df.tail(7)
#calls the last 7 rows
```

[5]:

	FEATURE_ID	FEATURE_NAME	FEATURE_CLASS	STATE_ALPHA	\
29529	2743558	Upland Community Church	Church	IN	
29530	2743605	John T. Myers Lock and Dam	Dam	KY	
29531	2745826	Kessinger Cemetery	Cemetery	IN	
29532	2748942	Rising Hope Baptist Church	Church	IN	
29533	2749894	Columbus City Cemetery	Cemetery	IN	
29534	2760566	Meadows Glen	Populated Place	IN	
29535	2760567	Meadows Knoll	Populated Place	IN	

	STATE_NUMERIC	COUNTY_NAME	COUNTY_NUMERIC	PRIMARY_LAT_DMS	\
29529	18	Grant	53	402756N	
29530	21	Union	225	374734N	
29531	18	Henry	65	395757N	
29532	18	Brown	13	391223N	
29533	18	Bartholomew	5	391256N	
29534	18	Hamilton	57	400407N	
29535	18	Hamilton	57	400416N	

	PRIM_LONG_DMS	PRIM_LAT_DEC	PRIM_LONG_DEC	SOURCE_LAT_DMS	\
29529	0853000W	40.465491	-85.499956	NaN	
29530	0875929W	37.792778	-87.991390	NaN	

```
29531      0851504W     39.965938     -85.251160            NaN
29532      0861329W     39.206422     -86.224714            NaN
29533      0855458W     39.215610     -85.916000            NaN
29534      0855913W     40.068638     -85.986989            NaN
29535      0855850W     40.070979     -85.980434            NaN

      SOURCE_LONG_DMS  SOURCE_LAT_DEC  SOURCE_LONG_DEC  ELEV_IN_M  ELEV_IN_FT  \
29529             NaN             NaN              NaN      282.0       925.0
29530             NaN             NaN              NaN       99.0       325.0
29531             NaN             NaN              NaN      341.0      1119.0
29532             NaN             NaN              NaN      213.0       699.0
29533             NaN             NaN              NaN      192.0       630.0
29534             NaN             NaN              NaN      241.0       791.0
29535             NaN             NaN              NaN      242.0       794.0

                MAP_NAME DATE_CREATED DATE_EDITED
29529  Hartford City West   12/26/2012         NaN
29530           Uniontown   12/28/2012  09/18/2013
29531     New Castle East   03/10/2013         NaN
29532           Nashville   07/16/2013         NaN
29533            Columbus   09/03/2013         NaN
29534           Riverwood   01/29/2014         NaN
29535           Riverwood   01/29/2014         NaN
```

[6]:
```
# TEST 1B
# Use one Pandas command to display the number of non-null observations
# and data type for every column.
df.info()
#prints summary of the dataframe

...
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29536 entries, 0 to 29535
Data columns (total 20 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   FEATURE_ID       29536 non-null  int64
 1   FEATURE_NAME     29536 non-null  object
 2   FEATURE_CLASS    29536 non-null  object
 3   STATE_ALPHA      29536 non-null  object
 4   STATE_NUMERIC    29536 non-null  int64
 5   COUNTY_NAME      29536 non-null  object
 6   COUNTY_NUMERIC   29536 non-null  int64
 7   PRIMARY_LAT_DMS  29536 non-null  object
 8   PRIM_LONG_DMS    29536 non-null  object
 9   PRIM_LAT_DEC     29536 non-null  float64
```

```
 10  PRIM_LONG_DEC    29536 non-null  float64
 11  SOURCE_LAT_DMS   3838 non-null   object
 12  SOURCE_LONG_DMS  3838 non-null   object
 13  SOURCE_LAT_DEC   3838 non-null   float64
 14  SOURCE_LONG_DEC  3838 non-null   float64
 15  ELEV_IN_M        29498 non-null  float64
 16  ELEV_IN_FT       29498 non-null  float64
 17  MAP_NAME         29536 non-null  object
 18  DATE_CREATED     29536 non-null  object
 19  DATE_EDITED      6237 non-null   object
dtypes: float64(6), int64(3), object(11)
memory usage: 4.5+ MB
```

[6]:
```
Ellipsis
```

Interpretation: Which columns immediately look like they may require special handling? Why?

1. Date Created and Date Edited need to be changed into dates before anything can be done with those variables.
2. Feature ID, State Numeric, County Numeric and these variables are stormed as numbers but they are really codes and identifiers, so these should be treated as categorical instead of numbers.
3. County Name and Map Name might have missing or inconsistent values so it would make sense to validate the date.
4. The source and prim coordinate values seem to contain nothing but null values, so for the sake of computational efficiency it might be worth removing these columns.

1.4 2. Selecting columns and rows — about 15 minutes

A DataFrame is two-dimensional. We often need only part of it.

1.4.1 Selecting one column

`df["COLUMN"]` returns a Pandas `Series`.

1.4.2 Selecting several columns

Pass a **list of column names**: `df[["A", "B"]]`.

1.4.3 `.loc[]`

`.loc[rows, columns]` is a flexible label-based selector.

[7]:
```
# One column -> Series
feature_names = df["FEATURE_NAME"]
print(type(feature_names))
feature_names.head()
```

```
<class 'pandas.core.series.Series'>
```

[7]:
```
0    Brouilletts Creek
1         Beaver Creek
2          Clear Creek
3           Coal Creek
4           Cole Creek
Name: FEATURE_NAME, dtype: object
```

[8]:
```
# Several columns -> DataFrame
location = df[["FEATURE_NAME", "COUNTY_NAME", "PRIM_LAT_DEC", "PRIM_LONG_DEC"]]
location.head()
```

[8]:

	FEATURE_NAME	COUNTY_NAME	PRIM_LAT_DEC	PRIM_LONG_DEC
0	Brouilletts Creek	Whitley	39.604762	-87.384782
1	Beaver Creek	Iroquois	40.969755	-87.813927
2	Clear Creek	Edgar	39.445869	-87.500300
3	Coal Creek	Edgar	39.641425	-87.501687
4	Cole Creek	Iroquois	40.632259	-87.558636

[9]:
```
# .loc[row selection, column selection]
# Here ':' means all rows.
df.loc[:, ["FEATURE_NAME", "FEATURE_CLASS", "ELEV_IN_M"]].head()
```

[9]:

	FEATURE_NAME	FEATURE_CLASS	ELEV_IN_M
0	Brouilletts Creek	Stream	144.0
1	Beaver Creek	Stream	186.0
2	Clear Creek	Stream	146.0
3	Coal Creek	Stream	151.0
4	Cole Creek	Stream	207.0

1.4.4 Student-led test 2

Create a DataFrame named `geo` containing only:

`FEATURE_NAME`, `FEATURE_CLASS`, `COUNTY_NAME`, `PRIM_LAT_DEC`, `PRIM_LONG_DEC`, and `ELEV_IN_M`.

Then display rows 10 through 15 using `.loc[]`.

Test yourself: Is the end label included when using `.loc[10:15]`? Determine this from your output rather than guessing.

[10]:
```
columns_needed = [
    # Complete this list
    "FEATURE_NAME",
    "FEATURE_CLASS",
    "COUNTY_NAME",
    "PRIM_LAT_DEC",
    "PRIM_LONG_DEC",
    "ELEV_IN_M"
]
```

```
geo = df[columns_needed]

# Display rows 10 through 15
...
geo.loc[10:15]
#Yes the end label is included when using .loc so 10:15 prints 6 rows as rows␣
 ↪10,11,12,13,14,15 are printed.
```

[10]:

	FEATURE_NAME	FEATURE_CLASS	COUNTY_NAME	PRIM_LAT_DEC	PRIM_LONG_DEC	ELEV_IN_M
10	Grand Calumet River	Stream	Cook	41.643924	-87.560878	177.0
11	Hawks Creek	Stream	Clark	39.383648	-87.516689	136.0
12	Hooper Branch	Stream	Iroquois	40.966700	-87.602811	192.0
13	Iroquois River	Stream	Kankakee	41.074199	-87.816429	181.0
14	Jonathan Creek	Stream	Edgar	39.866981	-87.500018	173.0
15	Jordan Creek	Stream	Vermilion	40.336148	-87.623077	197.0

1.5 3. Boolean filtering — about 20 minutes

Filtering asks Pandas to keep only rows satisfying a condition.

For example, `df["FEATURE_CLASS"] == "Stream"` does **not** immediately return streams. It first produces a Boolean Series of `True` and `False` values. That Boolean mask can then be used to select rows.

[11]:
```
is_stream = df["FEATURE_CLASS"] == "Stream"
is_stream.head(10)
```

[11]:
```
0     True
1     True
2     True
3     True
4     True
5    False
6     True
7     True
8     True
9     True
```

```
Name: FEATURE_CLASS, dtype: bool
```

[12]:
```
streams = df[is_stream]
streams[["FEATURE_NAME", "COUNTY_NAME", "ELEV_IN_M"]].head()
```

[12]:

	FEATURE_NAME	COUNTY_NAME	ELEV_IN_M
0	Brouilletts Creek	Whitley	144.0
1	Beaver Creek	Iroquois	186.0
2	Clear Creek	Edgar	146.0
3	Coal Creek	Edgar	151.0
4	Cole Creek	Iroquois	207.0

1.5.1 Predict first

What will the expression below return: a number, a Series, or a DataFrame?

```
(df["ELEV_IN_M"] > 300)
```

Make your prediction, then test it.

#I think that a series will be returned as we are comparing each value not necessary asking for a number or printing the dataframe.

[13]:
```
df["ELEV_IN_M"] > 300
```

[13]:
```
0        False
1        False
2        False
3        False
4        False
         …
29531     True
29532    False
29533    False
29534    False
29535    False
Name: ELEV_IN_M, Length: 29536, dtype: bool
```

1.5.2 Combining conditions

Use:

- `&` for AND
- `|` for OR
- `~` for NOT
- `.isin([...])` when matching several allowed values

Put each condition in parentheses.

[14]:
```
# Indiana streams above 250 m
high_in_streams = df[
    (df["STATE_ALPHA"] == "IN") &
    (df["FEATURE_CLASS"] == "Stream") &
    (df["ELEV_IN_M"] > 250)
]

high_in_streams[["FEATURE_NAME", "COUNTY_NAME", "ELEV_IN_M"]].head()
```

[14]:

	FEATURE_NAME	COUNTY_NAME	ELEV_IN_M
131	Alexandria Creek	Madison	258.0
202	Amity Branch	Hancock	253.0
210	Ams Run	Hamilton	253.0
237	Andy Run	Madison	263.0
255	Anthony Creek	Hancock	283.0

[15]:
```
# isin() is cleaner than writing many OR statements.
water_features = df[df["FEATURE_CLASS"].isin(["Stream", "Lake", "Reservoir",␣
 ↪"Canal"])]
water_features.head()
```

[15]:

	FEATURE_ID	FEATURE_NAME	FEATURE_CLASS	STATE_ALPHA	STATE_NUMERIC	\
0	403294	Brouilletts Creek	Stream	IN	18	
1	404056	Beaver Creek	Stream	IL	17	
2	406202	Clear Creek	Stream	IL	17	
3	406288	Coal Creek	Stream	IL	17	
4	406345	Cole Creek	Stream	IL	17	

	COUNTY_NAME	COUNTY_NUMERIC	PRIMARY_LAT_DMS	PRIM_LONG_DMS	PRIM_LAT_DEC	\
0	Whitley	183	393617N	0872305W	39.604762	
1	Iroquois	75	405811N	0874850W	40.969755	
2	Edgar	45	392645N	0873001W	39.445869	
3	Edgar	45	393829N	0873006W	39.641425	
4	Iroquois	75	403756N	0873331W	40.632259	

	PRIM_LONG_DEC	SOURCE_LAT_DMS	SOURCE_LONG_DMS	SOURCE_LAT_DEC	\
0	-87.384782	393919N	0874800W	39.655230	
1	-87.813927	410159N	0872342W	41.033090	
2	-87.500300	393514N	0874118W	39.587257	
3	-87.501687	393855N	0873750W	39.648646	
4	-87.558636	403756N	0873331W	40.632259	

	SOURCE_LONG_DEC	ELEV_IN_M	ELEV_IN_FT	MAP_NAME	DATE_CREATED	\
0	-87.799900	144.0	472.0	New Goshen	01/15/1980	
1	-87.395030	186.0	610.0	L'Erable	01/15/1980	
2	-87.688360	146.0	479.0	Dennison	01/15/1980	
3	-87.630580	151.0	495.0	Saint Bernice	01/15/1980	

```
4       -87.558636      207.0       679.0         Darrow   01/15/1980

  DATE_EDITED
0  11/19/2010
1         NaN
2         NaN
3         NaN
4         NaN
```

1.5.3 Student-led test 3A — basic filtering

Create `lakes` containing only records whose `FEATURE_CLASS` is `"Lake"`.

Then determine how many rows it contains. **Do not use a number supplied by the instructor. Let the data answer the question.**

[16]:
```
lakes = df[df["FEATURE_CLASS"] == "Lake"]
#Just a boolean mask and use it to only keep those rows that qualify.

# How many?
print("Number of lake records:", len(lakes))
#just code to count the number of rows also I keep deleting the ellipsis␣
 ↪because I like the formatting better without them.
```

```
Number of lake records: 803
```

1.5.4 Student-led test 3B — combine conditions

Find all records that:

1. are in Indiana,
2. are either `Lake` or `Reservoir`, and
3. have elevation greater than 250 m.

Store the result as `high_water`. Display only feature name, class, county, and elevation.

[17]:
```
#Make sure the location is in indiana, is a lake or resevoir and is more than␣
 ↪250m MSL
high_water = df[
    (df["STATE_ALPHA"] == "IN") &
    (df["FEATURE_CLASS"].isin(["Lake", "Resevoir"])) &
    (df["ELEV_IN_M"] >250)
]
# display only the feature name, class, county, and elevation
high_water[["FEATURE_NAME", "FEATURE_CLASS", "COUNTY_NAME", "ELEV_IN_M"]]
```

[17]:
```
        FEATURE_NAME FEATURE_CLASS COUNTY_NAME  ELEV_IN_M
76       Adams Lake          Lake    LaGrange      289.0
154      Allen Lake          Lake   Kosciusko      266.0
```

```
246         Lake Anne          Lake      Steuben      316.0
290     Appleman Lake          Lake     LaGrange      295.0
396       Atwood Lake          Lake     LaGrange      273.0
…                 …           …          …          …
22990     Indian Lake          Lake        Noble      276.0
22991   McClures Lake          Lake    Kosciusko      265.0
22992   Ridinger Lake          Lake    Kosciusko      258.0
24069     Clouse Lake          Lake        Noble      273.0
24190       Bell Lake          Lake      Johnson      251.0

[455 rows x 4 columns]
```

1.5.5 Check your reasoning

Explain why this expression can be dangerous:

```
df["FEATURE_CLASS"] == "Lake" | df["FEATURE_CLASS"] == "Reservoir"
```

What role do parentheses play when combining Pandas Boolean conditions?

They make it so that each comparison is evaluated for those conditions first, which ends up producing a boolean series for each condition and combining them with a &. Without the parentheses the operators would be applied in the wrong order.

1.6 4. Missing data — about 15 minutes

Real Earth-science datasets are rarely complete.

Useful tools:

- `.isna()` — identify missing values
- `.notna()` — identify non-missing values
- `.isna().sum()` — count missing values by column
- `.dropna()` — remove observations with missing values
- `.fillna()` — replace missing values

Important: missing-data treatment should follow the scientific question. Do not automatically drop every row containing any missing value.

[18]:
```
missing = df.isna().sum()
missing.sort_values(ascending=False)
```

[18]:
```
SOURCE_LONG_DMS    25698
SOURCE_LAT_DEC     25698
SOURCE_LONG_DEC    25698
SOURCE_LAT_DMS     25698
DATE_EDITED        23299
ELEV_IN_M             38
ELEV_IN_FT            38
```

```
FEATURE_ID            0
STATE_ALPHA           0
FEATURE_CLASS         0
FEATURE_NAME          0
STATE_NUMERIC         0
PRIM_LONG_DEC         0
PRIM_LAT_DEC          0
PRIM_LONG_DMS         0
PRIMARY_LAT_DMS       0
COUNTY_NAME           0
COUNTY_NUMERIC        0
MAP_NAME              0
DATE_CREATED          0
dtype: int64
```

1.6.1 Predict first

Consider:

`df.dropna()`

Would you expect this to retain most of the dataset or remove a large fraction? Use the missing-value summary above to justify your prediction **before** running it.

If I were to drop every row that would have at least one missing value then the majority of my data would disappear as at least 25698 of my 29535 rows are missing at least one value.

[22]:
```python
# Now test the prediction.
print("Original rows:", len(df))
print("Rows after dropna() on ALL columns:", len(df.dropna()))
```

```
Original rows: 29536
Rows after dropna() on ALL columns: 125
```

[23]:
```python
# A targeted approach: require elevation only.
elevation_data = df.dropna(subset=["ELEV_IN_M"])

print("Original rows:", len(df))
print("Rows with elevation:", len(elevation_data))
print("Rows removed:", len(df) - len(elevation_data))
```

```
Original rows: 29536
Rows with elevation: 29498
Rows removed: 38
```

1.6.2 Student-led test 4

Calculate the **percentage** of records with missing source latitude (`SOURCE_LAT_DEC`).

You know that a Boolean Series behaves like 1 for `True` and 0 for `False` when averaged. Try to use that idea.

Then answer: would filling missing source latitude with `0` be scientifically sensible? Why or why not?

No it would not be sensible as 0 is a possible value for a latitude, so it would end up poisoning any statistic made from this column with false data.

[26]:
```
missing_source_pct = df["SOURCE_LAT_DEC"].isna().mean() * 100
#isna() gives the series with true being a missing value, so I can use the␣
 ↪average with true being 1 and false being 0 to generate a fraction of␣
 ↪missing rows.
print(f"Missing source latitude: {missing_source_pct:.1f}%")
```

```
Missing source latitude: 87.0%
```

1.7 5. Counting, grouping, and aggregation — about 25 minutes

This is where Pandas becomes especially useful for scientific analysis.

1.7.1 `value_counts()`

Use this when you want frequencies of unique categories.

[27]:
```
df["FEATURE_CLASS"].value_counts().head(10)
```

[27]:
```
FEATURE_CLASS
Populated Place    3670
Cemetery           3627
Stream             3542
Church             3275
Canal              2535
School             2425
Building           2006
Civil              1673
Locale              970
Park                865
Name: count, dtype: int64
```

1.7.2 Student-led test 5A

Which five counties contain the largest number of records **for Indiana records only**?

You have already used filtering and `value_counts()`. Combine them.

[30]:
```
indiana = df[df["STATE_ALPHA"] == "IN"]
#value_counts() counts how often each county name appears and sorts from most␣
 ↪to least frequency.
```

```
indiana["COUNTY_NAME"].value_counts().head(5)
```

[30]:
```
COUNTY_NAME
Marion      2036
Lake        1463
Hamilton     721
Allen        651
Porter       580
Name: count, dtype: int64
```

1.7.3 groupby()

Conceptually, `groupby()` performs **split → apply → combine**:

1. split rows into groups;
2. apply a calculation to each group;
3. combine the group results.

Here we ask: *What is the mean elevation for each county?*

[31]:
```
county_elevation = (
    indiana.groupby("COUNTY_NAME")["ELEV_IN_M"]
           .mean()
)

county_elevation.head()
```

[31]:
```
COUNTY_NAME
Adams          251.441558
Allen          240.839506
Bartholomew    204.750903
Benton         230.568807
Blackford      271.658120
Name: ELEV_IN_M, dtype: float64
```

1.7.4 Predict first

What does each row in `county_elevation` represent?

I believe that it would represent the mean elevation, in meters, above mean sea level.

Now predict what changing `.mean()` to `.max()` would mean scientifically.

It would change it from an average elevation of the county to the elevation of the county's highest feature.

[32]:
```
indiana.groupby("COUNTY_NAME")["ELEV_IN_M"].max().head()
```

[32]:
```
COUNTY_NAME
Adams          279.0
```

```
Allen          302.0
Bartholomew    309.0
Benton         252.0
Blackford      286.0
Name: ELEV_IN_M, dtype: float64
```

1.7.5 agg() — several summaries at once

`agg()` lets us apply multiple summary functions to each group.

[33]:
```
feature_elevation = (
    indiana.groupby("FEATURE_CLASS")["ELEV_IN_M"]
           .agg(["count", "mean", "median", "min", "max"])
)

feature_elevation.head(10)
```

[33]:

FEATURE_CLASS	count	mean	median	min	max
Airport	585	233.323077	242.0	113.0	348.0
Arch	4	184.500000	189.5	153.0	206.0
Area	2	186.500000	186.5	133.0	240.0
Bar	5	146.800000	141.0	116.0	175.0
Basin	12	165.500000	148.0	130.0	209.0
Bay	11	215.727273	197.0	175.0	273.0
Beach	3	211.000000	218.0	179.0	236.0
Bend	29	162.655172	153.0	109.0	246.0
Bridge	111	182.315315	173.0	107.0	292.0
Building	2001	222.750125	227.0	110.0	359.0

1.7.6 Student-led test 5B

Using `groupby()` and `agg()`, calculate the following elevation statistics **by county**:

- count
- mean
- standard deviation
- minimum
- maximum

Hint: the aggregation name for standard deviation is `"std"`.

[34]:
```
#splits up the rows, selectis my column of interest, and applies the function␣
 ↪to each group.
county_stats = indiana.groupby("COUNTY_NAME")["ELEV_IN_M"].agg(
    ["count", "mean", "std", "min", "max"]
)
county_stats.head()
```

[34]:

COUNTY_NAME	count	mean	std	min	max
Adams	231	251.441558	7.063366	232.0	279.0
Allen	648	240.839506	10.829243	176.0	302.0
Bartholomew	277	204.750903	20.688670	173.0	309.0
Benton	109	230.568807	10.971653	211.0	252.0
Blackford	117	271.658120	9.651838	204.0	286.0

1.7.7 Student-led test 5C — make a scientific comparison

Compare elevation statistics for only `Stream`, `Lake`, and `Summit` features.

Do this in two stages:

1. filter the rows;
2. group by `FEATURE_CLASS` and calculate `count`, `mean`, `min`, and `max`.

Which class has the highest mean elevation?

[39]:
```
#filter to only keep features of interest
selected = indiana[indiana["FEATURE_CLASS"].isin(["Stream", "Lake", "Summit"])]
#group by class and calculate elevation stats
comparison = selected.groupby("FEATURE_CLASS")["ELEV_IN_M"].agg(
    ["count", "mean", "min", "max"]
)
comparison
# which class has the highest mean elevation?
print("Highest mean elevation:", comparison["mean"].idxmax())
```

```
Highest mean elevation: Lake
```

1.8 6. Sorting and creating new variables — about 15 minutes

1.8.1 `sort_values()`

Sorting is often the last step after calculating a quantity.

[37]:
```
# Ten highest-elevation Indiana features
highest = (
    indiana.sort_values("ELEV_IN_M", ascending=False)
           [["FEATURE_NAME", "FEATURE_CLASS", "COUNTY_NAME", "ELEV_IN_M"]]
           .head(10)
)

highest
```

[37]:

	FEATURE_NAME	FEATURE_CLASS	COUNTY_NAME	ELEV_IN_M
24104	Hoosier Hill	Summit	Wayne	382.0

```
9606   Mount Gilliard Cemetery         Cemetery    Randolph      374.0
940                     Bethel  Populated Place       Wayne      369.0
2192                    Carlos  Populated Place    Randolph      369.0
6504           Hopewell Church           Church       Wayne      368.0
23282   Township of Greensfork            Civil    Randolph      366.0
19721                Snow Hill  Populated Place    Randolph      363.0
294                       Arba  Populated Place    Randolph      363.0
1266              Bloomingport  Populated Place    Randolph      363.0
3181                     Crete  Populated Place    Randolph      362.0
```

1.8.2 Creating a derived column

Scientific datasets often contain variables from which we calculate new quantities.

This dataset already contains elevation in meters and feet. We can use that redundancy to test a conversion.

[38]:
```
# Make a copy before adding experimental/derived columns.
work = indiana.copy()

work["ELEV_FT_CALCULATED"] = work["ELEV_IN_M"] * 3.28084
work["ELEV_FT_DIFFERENCE"] = work["ELEV_IN_FT"] - work["ELEV_FT_CALCULATED"]

work[["ELEV_IN_M", "ELEV_IN_FT", "ELEV_FT_CALCULATED", "ELEV_FT_DIFFERENCE"]].
 ↪head()
```

[38]:
```
    ELEV_IN_M  ELEV_IN_FT  ELEV_FT_CALCULATED  ELEV_FT_DIFFERENCE
0       144.0       472.0            472.44096            -0.44096
5       241.0       791.0            790.68244             0.31756
20      108.0       354.0            354.33072            -0.33072
27      148.0       486.0            485.56432             0.43568
34      194.0       636.0            636.48296            -0.48296
```

1.8.3 Student-led test 6

Use Pandas to answer both questions:

1. What is the largest **absolute** difference between the supplied feet elevation and your calculated value?
2. Why might the difference not be exactly zero? The difference is not exactly zero because it would appear that the supplied elevations are rounded to the nearest foot whereas the calculated values are not.

Hint: a Pandas Series has an `.abs()` method.

[40]:
```
#calculates the absolute difference between the supplied feet and the␣
 ↪calculated feet, removing signs, and skipping null values.
max_abs_difference = work["ELEV_FT_DIFFERENCE"].abs().max()
max_abs_difference
```

[40]:
```
np.float64(0.5039600000000064)
```

1.9 7. Dates in Pandas — about 10 minutes

`DATE_CREATED` was initially read as text. To perform date-aware operations, convert it with `pd.to_datetime()`.

`errors="coerce"` means values that cannot be interpreted as dates become `NaT` (Not-a-Time) rather than stopping the program.

[41]:
```
work["DATE_CREATED"] = pd.to_datetime(work["DATE_CREATED"], errors="coerce")
work["DATE_EDITED"] = pd.to_datetime(work["DATE_EDITED"], errors="coerce")

work[["DATE_CREATED", "DATE_EDITED"]].dtypes
```

[41]:
```
DATE_CREATED    datetime64[ns]
DATE_EDITED     datetime64[ns]
dtype: object
```

[42]:
```
# The .dt accessor exposes datetime components.
work["YEAR_CREATED"] = work["DATE_CREATED"].dt.year

work[["FEATURE_NAME", "DATE_CREATED", "YEAR_CREATED"]].head()
```

[42]:

	FEATURE_NAME	DATE_CREATED	YEAR_CREATED
0	Brouilletts Creek	1980-01-15	1980
5	Concordia Theological Seminary	1980-01-15	1980
20	Lost Lake	1980-01-15	1980
27	Spring Creek	1979-02-14	1979
34	Singleton Ditch	1999-09-30	1999

1.9.1 Student-led test 7

Answer these with Pandas:

1. What is the earliest creation date in the dataset?
2. What is the latest creation date?
3. How many records were created in each year?
4. Which creation year has the most records?

For #3, try `value_counts()` on `YEAR_CREATED` and then decide whether sorting by **count** or by **year** better answers the question you are asking.

[43]:
```
#now that dates have been created in datetime values min simply returns the␣
 ↪smallest(aka earliest) and max returns the largest(aka latest).
earliest = work["DATE_CREATED"].min()
latest = work["DATE_CREATED"].max()
```

```
print("Earliest:", earliest)
print("Latest:", latest)

# value counts the amount of entries per year created the highest number would␣
 ↪be most busy.
counts_by_count = work["YEAR_CREATED"].value_counts()
print(counts_by_count.head(10))
#resorts the years and returns the amount of entries for each year
counts_by_year = work["YEAR_CREATED"].value_counts().sort_index()
print(counts_by_year)
#returns the label of the index with the largest value and the count of entries␣
 ↪in that year.
top_year = counts_by_year.idxmax()
print("Year with the most records:", top_year)
print("Number of records that year:", counts_by_year.max())
```

```
Earliest: 1979-02-14 00:00:00
Latest: 2014-01-29 00:00:00
YEAR_CREATED
1979    18199
1982     4947
2011     1649
1995     1017
2007      914
2008      711
2012      516
2006      388
2010      349
1992      348
Name: count, dtype: int64
YEAR_CREATED
1979    18199
1980        5
1982     4947
1990        2
1991       84
1992      348
1993      144
1994        7
1995     1017
1996        4
1997        5
1998       22
1999       17
2000        6
2001       15
```

```
2002        3
2003        5
2004       11
2005        6
2006      388
2007      914
2008      711
2009       38
2010      349
2011     1649
2012      516
2013        3
2014        2
Name: count, dtype: int64
Year with the most records: 1979
Number of records that year: 18199
```

---

2 8. Final challenge — about 20 minutes

2.1 From Pandas syntax to data analysis

Work individually or in pairs. You may use earlier sections of this notebook, but your final code should run from top to bottom.

Using **Indiana records only**, answer the following:

1. What are the five most common feature classes?
2. Which five counties contain the greatest number of features?
3. For `Stream`, `Lake`, and `Summit`, calculate the number of observations and mean, minimum, and maximum elevation.
4. Identify the 10 highest-elevation features and report their name, feature class, county, and elevation.
5. What percentage of records are missing source coordinates?
6. **Develop one additional question that can be answered with this dataset.**

For Question 6, include:

- your question;
- the Pandas operations you chose;
- your result; and
- a 2–3 sentence interpretation.

2.1.1 Constraint

For at least one question, your solution must combine **three or more Pandas operations** (for example: filter → group → aggregate → sort).

[47]:
```
# FINAL CHALLENGE - Question 1
# Five most common feature classes in Indiana
indiana = df[df["STATE_ALPHA"] == "IN"]
indiana["FEATURE_CLASS"].value_counts().head(5)
```

[47]:
```
FEATURE_CLASS
Populated Place    3667
Cemetery           3627
Stream             3468
Church             3274
Canal              2528
Name: count, dtype: int64
```

[48]:
```
# FINAL CHALLENGE - Question 2
# Five counties with the greatest number of features
indiana["COUNTY_NAME"].value_counts().head(5)
```

[48]:
```
COUNTY_NAME
Marion      2036
Lake        1463
Hamilton     721
Allen        651
Porter       580
Name: count, dtype: int64
```

[49]:
```
# FINAL CHALLENGE - Question 3
# Stream/Lake/Summit elevation summary
#combining my three operations
indiana[indiana["FEATURE_CLASS"].isin(["Stream", "Lake", "Summit"])] \
    .groupby("FEATURE_CLASS")["ELEV_IN_M"] \
    .agg(["count", "mean", "min", "max"])
```

[49]:

FEATURE_CLASS	count	mean	min	max
Lake	794	241.539043	101.0	344.0
Stream	3468	205.638120	99.0	356.0
Summit	298	240.718121	107.0	382.0

[50]:
```
# FINAL CHALLENGE - Question 4
# Ten highest-elevation Indiana features
#sort descending
indiana.sort_values("ELEV_IN_M", ascending=False)[
    ["FEATURE_NAME", "FEATURE_CLASS", "COUNTY_NAME", "ELEV_IN_M"]
].head(10)
```

[50]:

	FEATURE_NAME	FEATURE_CLASS	COUNTY_NAME	ELEV_IN_M
24104	Hoosier Hill	Summit	Wayne	382.0

```
9606   Mount Gilliard Cemetery         Cemetery    Randolph      374.0
940                     Bethel  Populated Place       Wayne      369.0
2192                    Carlos  Populated Place    Randolph      369.0
6504           Hopewell Church           Church       Wayne      368.0
23282   Township of Greensfork            Civil    Randolph      366.0
19721                Snow Hill  Populated Place    Randolph      363.0
294                      Arba  Populated Place    Randolph      363.0
1266             Bloomingport  Populated Place    Randolph      363.0
3181                    Crete  Populated Place    Randolph      362.0
```

[51]:
```
# FINAL CHALLENGE - Question 5
# Percentage missing source coordinates
# Decide how you will define "missing source coordinates".
# State that definition in a comment before your calculation.

#A missing source coordinate is one that has a value of NaN or null.
missing_pct_indiana = indiana["SOURCE_LAT_DEC"].isna().mean() * 100
print(f"Indiana records missing source coordinates: {missing_pct_indiana:.1f}%")
```

```
Indiana records missing source coordinates: 87.2%
```

2.1.2 FINAL CHALLENGE — Question 6: Your own question

My question: Among the streams of Indiana, which five counties, with at least 20 streams, have the highest median elevation.

Why this question is useful/interesting: It provides an idea of how the geography varies by county where the northeast might have a larger value than the southwest for example.

Pandas operations I expect to need: filter, groupby, aggregate, filter on the aggregated result, and sort.

[52]:
```
# Write your analysis for Question 6 here.
# Filter to IN with known elevation values
streams = indiana[(indiana["FEATURE_CLASS"] == "Stream")].
 ↪dropna(subset=["ELEV_IN_M"])

#Generate my groups, counts, and median elevation
by_county = streams.groupby("COUNTY_NAME")["ELEV_IN_M"].agg(["count", "median"])

# Drop counties with less than 20 streams
by_county = by_county[by_county["count"] >= 20]

# Sort top five descending
by_county.sort_values("median", ascending=False).head(5)
```

[52]:
```
             count  median
COUNTY_NAME
Randolph        78   324.0
```

Wayne	144	312.5
Henry	120	303.5
Union	34	277.0
Jay	64	276.0

Interpretation (2–3 sentences): The five highest median stream elevation counties in Indiana are all clustered on the central Eastern border with Ohio. This makes sense due to the geography of the area not being shaped by any major rivers, so any streams will be very gradual on what is already high ground relative to the rest of the state. This makes sense to me as streams in the far south would be influenced by the large Ohio River Valley.

2.2 Exit ticket — 5 minutes

Answer without copying definitions from above.

1. What is the difference between `value_counts()` and `groupby()`? Value counts simply counts how many times a value appears in a column, whereas groupby splits the data into groups and calculates various stats within these groups.
2. Why can `dropna()` be dangerous when used without specifying a subset? It will remove every row with a missing value.
3. What does a Boolean mask contain? A series of true or false values
4. Write, in words, the sequence of Pandas operations you would use to find the county with the highest mean elevation among streams only. I would filter for my class and group them by county, taking the mean and then sorting for the highest value.
5. Which Pandas operation from today do you think will be most useful for your own research data, and why? From my experience dropna has been my most used piece of pandas code simply because hurricane datasets are awful and can have tens of millions of missing values that add several gigabytes to a file. So the computational benefits of dropna are indispensible.

2.3 Optional extension if you finish early

Use `PRIM_LAT_DEC` and `PRIM_LONG_DEC` to make a simple scatter plot of Indiana feature locations. Color or separate the points by one feature class of your choice.

You may use Pandas' built-in `.plot.scatter()` or Matplotlib. This extension is intentionally **not scaffolded**: decide what subset you want to visualize and what question the figure should answer.